

Use Case: AI Knowledge Assistant

1. Background & Objective

When new engineers join, one of the biggest hurdles is finding information — whether it's about our systems, processes, or project setups. Docs are everywhere, and “where do I find this?” becomes a daily question.

To solve that, this onboarding project focuses on building a small but functional AI Knowledge Assistant — a system that can read and understand internal content (synthetic in this case), answer questions about it, and provide references to where it found the answers.

The goal is to learn how real-world AI solutions are built — how data flows from raw documents to embeddings, retrieval, large language model (LLM) reasoning, and finally to a usable chat interface.

By the end of the project, you should be comfortable connecting the dots between:

- AI models (embeddings, language models)
- Retrieval pipelines (RAG-style)
- Data ingestion and exports
- System design aligned with Azure/AWS-like architectures

2. What You'll Build

You'll create a local, open-source-based AI assistant that:

- Reads knowledge documents (Markdown, PDFs, DOCX, CSVs)
- Generates synthetic data to simulate an organization's knowledge base
- Builds embeddings and stores them in a local vector database
- Answers natural language questions with citations
- Lets users export responses and references to Word or Excel
- Tracks usage metrics like response time and retrieval quality

Think of it as your own private ChatGPT for internal knowledge.

3. Key Functional Areas

Area	Description
Ingestion	Load, clean, and normalize documents from local folders
Synthetic Data Generator	Script to produce mock company data — policies, FAQs, guides, SOPs
Chunking & Indexing	Break text into chunks and embed them into a vector store
Retrieval & Response	Fetch top relevant chunks, pass them to an LLM, and generate grounded answers
Interface	Simple web UI for querying and viewing citations
Evaluation & Logging	Capture metrics like latency, recall, and grounding

4. Technical Foundations

Everything runs locally or on a free-tier-equivalent setup using **open-source tools only**. Technologies are chosen to mirror how the same architecture would scale in Azure or AWS later.

Layer	Suggested Tools (Local / Open Source)	Maps To (Cloud Equivalent)
LLM / Chat	Ollama (Llama 3.1+, Qwen 2.5, Mistral, Mixtral)	Azure OpenAI / AWS Bedrock
Embeddings	Sentence-Transformers (bge-small-en-v1.5, MiniLM)	Azure Embeddings / Titan
Vector DB	Chroma or FAISS (HNSW indexing where supported)	Cognitive Search / Kendra
API	FastAPI or Express	Azure Functions / Lambda
UI	Streamlit or Next.js	Web App / CloudFront

Layer	Suggested Tools (Local / Open Source)	Maps To (Cloud Equivalent)
Parsing	PyMuPDF, docx2txt, pandas, unstructured	Textract / Document Intelligence
Storage	SQLite or PostgreSQL	Blob Storage / S3
Exports	python-docx, openpyxl	OpenXML
Monitoring	JSON logs, Matplotlib, structured trace IDs	App Insights / CloudWatch

5. Tentative Task Breakdown

Below is a broad set of tasks — use these as reference points for splitting tickets.

A. Project Setup

- Create repository and folder structure (core, ui, api, data, tools)
 - Add .env, requirements.txt, and starter scripts
 - Document local setup steps clearly
-

B. Synthetic Data Generation

- Write a script (synth_data.py) that generates sample content across themes:
 - Company overview and team FAQs
 - Technical tutorials or API snippets
 - SOPs and internal best practices
 - Each document includes metadata JSON:
 - title, tags, type, date
-

C. Ingestion & Normalization

- Build parsers for Markdown, PDF, DOCX, CSV

- Clean text (remove headers, blank lines, special characters)
 - Split documents into configurable chunks (e.g., 500–1000 tokens, optional overlap)
 - Store processed text and metadata locally
-

D. Embeddings & Vector Index

- Generate embeddings using sentence-transformers
 - Index chunks using Chroma or FAISS (local persistence)
 - Expose a similarity search function to retrieve top-k chunks
-

E. Retrieval-Augmented Generation

Implement a query flow:

1. Receive user question
2. Retrieve top-k relevant chunks
3. Optionally rerank results using a local cross-encoder
4. Construct a prompt using:
 - System instructions
 - Developer prompt
 - Retrieved context
 - User question
5. Generate an answer using a local LLM
6. Return answer with citations

Expose this flow via an API endpoint (/ask).

F. Interface

- Build a simple chat-style UI (Streamlit or Next.js)
- Include:

- Question input
 - Streaming or incremental answer display
 - Expandable citations
 - Export buttons (Word / Excel)
 - Manual re-index trigger
-

G. Evaluation & Logging

- Log each query with:
 - Timestamp
 - Latency
 - Retrieved chunk IDs
 - Maintain a small evaluation question set
 - Measure:
 - Recall@k
 - Citation coverage
 - Answer grounding (overlap with retrieved context)
 - Visualize results using Matplotlib or summary tables
-

H. Security & Compliance

- Store secrets in local .env
 - Protect API endpoints with basic auth or token checks
 - Implement regex-based redaction for emails, phone numbers, IDs
 - Include a short note on ethical and safe AI usage
-

I. Packaging & Handoff

- Finalize README and architecture diagram

- Provide demo scripts:
 - make reindex
 - make run
 - make eval
 - Optional: short walkthrough video or slides
-

6. Expected Outcome

By completing this project, you should be able to:

- Explain how a RAG pipeline works end-to-end
 - Run and extend an AI assistant locally
 - Understand how the same design scales in cloud environments
 - Demonstrate practical skills in embeddings, LLMs, and retrieval
 - Present the system as a mini internal knowledge assistant
-

7. Optional Extensions

- Hybrid retrieval (keyword + vector)
- Local reranking models for improved precision
- Usage dashboards (queries over time, latency)
- Tool-calling patterns (reindex, export, evaluate as tools)
- Docker-based local deployment